
A Container based Networking Digital Twin

Yongsen Fan

Student Number: 22131565

Supervisor: Miguel Rio

Abstract

Network digital twins (NDTs) promise safer, repeatable network experimentation, but evidence is often divided among architecture proposals, emulation studies, topology evaluations, and automated-control demonstrations. This study evaluates a scenario-driven, reproducibility-oriented Docker NDT using validated scenarios, deterministic routing, Linux traffic control, ping and iperf3 measurements, and structured evidence retention. The evaluation covers controlled bandwidth, one-way delay and loss, direct and multi-router topologies, Germany50 scaling, artificial-intelligence (AI)-assisted scenario generation, and reinforcement-learning (RL) control. Fifteen valid fixed-block bandwidth runs, five each at configured rates of 20, 50, and 100 Mbps, produced mean throughputs of 19.20, 47.80, and 94.76 Mbps. Configured one-way delays of 10, 30, and 50 ms produced mean round-trip times (RTTs) of 27.008, 80.303, and 133.686 ms across five runs per condition. The complete 50-node/88-link Germany50 topology was instantiated, but its 4,224-entry route plan was validated only in dry-run mode and real Docker traffic covered three representative paths, not all pairs. The official OpenAI request returned HTTP 429, whereas a separate Qwen-compatible request passed validation and controlled execution. Across 80 valid real-Docker measurements from four control policies, the threshold heuristic achieved a higher mean reward than the evaluated Q-learning configuration. The results support an artifact-backed experimental workflow in the tested configurations while exposing limits in host fidelity, topology coverage, provider availability, evidence preservation, and learning-based control performance.

1 Introduction

Modern networks need safe ways to test configuration and control changes before deployment. A network digital twin (NDT) provides a digital environment in which network models, state, and control interfaces can be exercised. Reference architectures describe NDT components and closed-loop management (Zhou et al., 2026; International Telecommunication Union, 2022); data-driven studies examine routing and the trade-off among fidelity, efficiency, and flexibility (Almasan et al., 2022; Hui et al., 2023).

The practical problem is narrower than reproducing a physical network. An experimental twin must translate a scenario into controlled execution, verify routes and impairments, measure traffic, and retain the artifacts needed to audit the run. Network emulators and reproducibility toolchains provide strong foundations (Lantz et al., 2010; Handigol et al., 2012; Gallenmüller et al., 2021). However, the closest work does not quantify this particular integration: whether one bounded Docker pipeline can enforce and measure declared conditions while being reused for larger topologies and guarded automation.

I evaluate that focused gap with a scenario-driven Docker NDT. JavaScript Object Notation (JSON) scenarios enter validation before Docker topology construction, deterministic routing, Linux traffic control, ping and iperf3 measurement, evidence serialization, and scoped cleanup. The same guarded pipeline is then exercised on a Germany50-derived topology, artificial-intelligence (AI)-assisted scenario generation, measured closed-loop control, and a bounded supporting Dashboard. The evaluation concerns the tested container environment, not equivalence with a physical network.

1.1 Research gap and questions

The gap is a lack of evidence about whether a bounded, scenario-driven Docker NDT can both enforce and measure controlled network conditions and reuse one validated execution-and-evidence pipeline as topology and guarded automation demands increase. This single integration question is specific, testable against retained measurements and artifacts, and limited to the evaluated configurations.

The main research question is:

To what extent can a bounded, scenario-driven Docker network digital twin enforce and measure controlled network conditions while reusing one validated execution-and-evidence pipeline across increasingly complex topologies and guarded automation tasks?

It is decomposed into three questions:

- RQ1:** How closely and consistently do measured behaviours follow configured bandwidth, delay, and packet-loss conditions?
- RQ2:** How far can the validated pipeline extend from direct and multi-router cases to Germany50 within the retained routing and traffic evidence?
- RQ3:** Can guarded AI-assisted generation and closed-loop control reuse the pipeline, and what limits appear under explicit checks and comparators?

Within RQ3, RL is only a bounded closed-loop path-selection use case. It tests whether prior digital-twin measurements can inform a controller while all policies reuse the same execution-and-evidence pipeline. The contribution is the integration and bounded evaluation of RL-based control, not a new RL algorithm or a claim of superiority over the heuristic.

1.2 Contributions

The first contribution is a validated Docker pipeline joining scenario checks, topology construction, routing, impairment, measurement, cleanup, and evidence retention. The second is a bounded quantitative evaluation across bandwidth, one-way delay, packet loss, multi-router execution, and Germany50 structural scale. The third evaluates guarded scenario generation and integrates RL-based closed-loop control with the real-Docker pipeline for bounded comparison against transparent policies. Traceability is an audit property of the method—the link among inputs, commands, measurements, and retained artifacts—rather than a separately scored performance outcome. Germany50 real traffic covers three representative paths, the complete route plan is dry-run evidence, provider outcomes remain separate, and the Dashboard is only a supporting interface.

2 Background and Related Work

2.1 Network digital twins

The Internet Research Task Force (IRTF) draft organizes NDT data, models, mappings, and interfaces, but is a work in progress rather than a standard (Zhou et al., 2026). International Telecommunication Union Telecommunication Standardization Sector (ITU-T) Recommendation Y.3090 specifies requirements and a layered closed-loop architecture (International Telecommunication Union, 2022). Almasan et al. (2022) demonstrate quality-of-service (QoS)-aware routing, while Hui et al. (2023) analyze fidelity, efficiency, and flexibility. These sources define the concept and model-centric opportunities; they do not determine the measured behaviour of this container implementation.

2.2 Closest emulation and reproducibility systems

Mininet runs hosts, switches, links, and real application code on one machine for software-defined networking (SDN) experiments (Lantz et al., 2010). Handigol et al. (2012) add runnable specifications and complete artifacts to container-based emulation. CORE supports real-time wired and wireless emulation, distributed execution, and connection to physical networks (Ahrenholz et al., 2008). These systems establish broad topology execution; this study instead evaluates controlled Docker measurements through one specific scenario-to-evidence path.

The closest Docker systems cover substantial emulation capability. MeDICINE uses Containernet to prototype containerized network functions across multiple points of presence (PoPs), with resource limits, link impairments, and orchestration interfaces (Peuster et al., 2016). Kathará provides lightweight container-based network scenarios and reports comparative scalability and performance (Scazzariello et al., 2020). Their topology and container support is broader than the small forms tested here, so this dissertation does not claim a stronger emulator. Its question concerns integration of declared impairments, measured traffic, retained per-run evidence, and guarded AI/control extensions in one bounded Docker NDT.

Reproducibility is a separate capability from emulation breadth. Handigol et al. (2012) package executable experiments and artifacts; the pos framework scripts orchestration, measurement, evaluation, and artifact bundling for repeatable network experiments (Gallenmüller et al., 2021). pos is the closest workflow comparator, but it is a general experiment methodology rather than an evaluation of this Docker NDT’s impairment, topology, and guarded-automation pipeline. Appendix Table 6 compares topology, Docker support, impairment, artifacts, scale, and guarded automation directly.

2.3 Impairment, topology, and automation

Linux queueing discipline (qdisc) `netem` can impose delay and loss, but timer granularity, Transmission Control Protocol (TCP) queues, and qdisc placement influence observations (iproute2 Project, n.d.). Configured one-way conditions are therefore reported separately from ping round-trip time (RTT), end-to-end loss, and `iperf3` throughput.

The Internet Topology Zoo supplies operator-derived graphs for structural studies (Knight et al., 2011). Germany50 provides 50 nodes and 88 links here, but a graph alone proves neither installed routes nor exercised traffic. The evaluation separates instantiation, full route-plan dry-run validation, and selected-path real traffic.

Large language models (LLMs) have been studied for network configuration and management. LLM-NetCFG verifies generated configurations before application, and a broader review identifies reliability, security, and evaluation risks (Lira et al., 2025; Liu et al., 2025). These risks motivate schema validation, semantic checks, allowlisted projection, and restricted execution.

Reinforcement learning (RL) can learn repeated control decisions. DeepRM compares learned resource management with heuristics (Mao et al., 2016); NDT studies use evolutionary strategies or twin assistance to improve learning and safety (Güemes-Palau et al., 2022; Cheng et al., 2025). These results justify testing a learner against transparent policies, but do not predict superiority in the present small, ordered task.

2.4 Gap synthesis

Closest systems already support network emulation, Docker nodes, scaling, or reproducible experiment packaging. What remains unanswered is the main RQ for this bounded integration: whether one validated Docker execution-and-evidence pipeline can enforce and measure controlled conditions, then be reused across the tested topology and guarded automation extensions. The evaluation answers only that question and reports failures, confounds, and evidence limits alongside successful observations.

3 System Design and Methodology

3.1 Objectives and scope

The design starts from explicit, machine-validated scenarios and uses container network stacks with standard Linux tools. Routing and impairment state are checked before measurements are trusted, and each run is expected to leave structured evidence. Extensions reuse this bounded execution path. The system is an experimental NDT, not a synchronized replica of an operational network; its claims concern controlled behaviour in the tested configurations.

Figure 1 summarizes the execution path. A scenario/configuration enters validation before any topology is built. The validated representation drives Docker topology creation, routing and forwarding, impairment configuration, and measurement. Results are serialized as JSON and comma-separated value (CSV) files and accompanied where available by logs, summaries, statistics, and plots. Cleanup is scoped to project-owned resources so that a failed run does not justify broad container or network deletion.

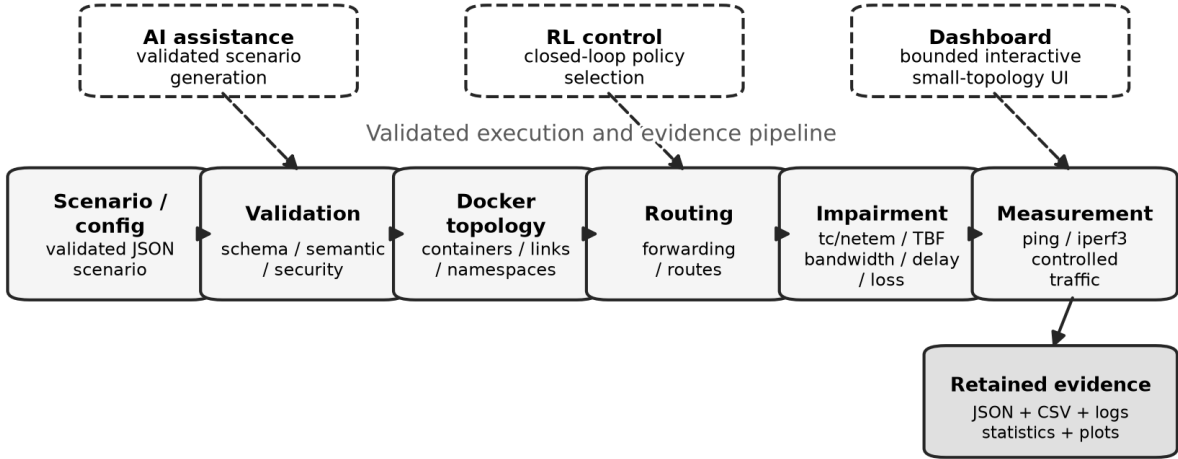


Figure 1: Overall architecture. Scenario validation, Docker execution, routing, impairment, measurement, and evidence retention form the core pipeline. AI assistance, RL control, and the Dashboard are bounded extensions rather than independent core systems.

3.2 Scenario representation and validation

Scenarios describe node roles, links, addressing, impairments, and traffic endpoints. The method follows a fixed chain: parse; validate schema, semantics, and bounds; build project-scoped Docker resources; configure and verify forwarding, routes, and qdiscs; run measurements; serialize metrics and logs; then remove run-scoped resources. Small experiments use allowlisted direct, routed, or two-router forms; the generic path supports larger graphs. AI-originated content also passes prompt constraints and projection, so it cannot choose commands, images, credentials, or host paths. Appendix A.3 gives a valid scenario and the implemented checks.

3.3 Docker topology and routing

Docker containers attach to one deterministic network per link, and router containers enable forwarding. The generic path allocates point-to-point subnets, computes breadth-first shortest paths with deterministic tie-breaking, installs static routes, and checks the expected destination route before traffic. Appendix A.3 specifies addressing, route scope, commands, and assumptions.

Germany50 extends this mechanism using imported Topology Zoo data. The normalized scenario contains 50 router nodes and 88 links with deterministic subnets. Two routing modes were retained. Selected mode installs the routes needed for the three representative endpoint pairs. Full mode generates the theoretical 4,224-entry plan, but the retained full-mode evidence is a dry-run validation only. This separation is fundamental: plan generation, route installation, and traffic exercise are distinct claims.

3.4 Impairment and measurement pipeline

Linux traffic control implements configured conditions. A token bucket filter (TBF) is used for bandwidth-only links; combined bandwidth and delay/loss use the implemented `netem rate` command, not the same TBF hierarchy. The framework checks qdisc state and runs ping and reverse-TCP iperf3. Ping supplies counts, loss, and RTT; iperf3 supplies throughput. Configured one-way conditions therefore remain separate from measured end-to-end quantities. Exact templates appear in Appendix A.3.

Each run writes its normalized scenario and metrics into a run-specific directory. Batch runners add manifests, per-run rows, summaries, environment records, and cleanup outcomes. Failed or retried attempts remain visible. Analysis assumes that retained tool output represents the container path exercised by its runner; it does not assume synchronized physical-network state, identical host timing, or unmeasured all-pairs reachability. The dissertation inventory distinguishes repository-reported values from statistics recomputed during a read-only audit.

3.5 Automation extensions

AI-assisted generation. The AI path requests a structured scenario, validates it, projects it into the executable representation, and permits only dry-run or controlled execution. Provider identity is retained separately from validation outcome. The official OpenAI call and the OpenAI-compatible Qwen call are recorded as different evidence items.

Closed-loop control. The RL extension uses a minimal dual-path Docker topology. All four policies reuse the same Docker execution, route-verification, measurement, and evidence pipeline. After initialization, only Q-learning uses the prior measured RTT, loss, throughput, and current path as state and updates a Q-table from measured reward. The threshold heuristic selects from the known phase configuration, while fixed A/B always select their named paths. Appendix A.6 records the exact formulation and execution provenance.

Dashboard. The reviewer-facing `dashboard/interactive_server.py` is a supporting interface, not the front end for every formal experiment. It permits dry-runs for direct, routed, and two-router scenarios; real Docker execution is restricted to direct topology, with routed and two-router requests rejected by both browser and server. It retains parameter bounds, restricted artifact downloads, and run-scoped cleanup. Germany50 and RL remain read-only results, and arbitrary Docker commands are not accepted.

3.6 Reproducibility and security

The sealed formal evidence contains versioned scenarios, runners, per-run metrics, summaries, figures, failures, and cleanup records. GitHub commit `5f0d4962` is the intermediate reviewer-readiness software baseline: its two scripts provide fresh-clone setup, image build, 69 unit tests, and Dashboard start without invoking formal matrices. The balanced bandwidth cohort records `4402f85f` as its execution commit. The final code/evidence snapshot is `09a14859`, linked in Appendix A.8. This chronology separates reviewer tooling, experimental execution, and publication. Timing remains host-sensitive because Docker shares its kernel and resources with the host.

4 Experimental Methodology

4.1 Environment, metrics, and statistics

The retained experiments ran in the project’s Docker/Windows Subsystem for Linux (WSL) environment using project container images, Python runners, Linux traffic control, ping, and iperf3. Exact host hardware utilization was not retained consistently, so hardware-level performance generalization is avoided. Every reported result is tied to a run directory or repository summary in the evidence inventory.

Version evidence is available for the dissertation-primary balanced bandwidth cohort. Its retained `runs/bandwidth-balanced-evidence-supplement-20260826/planned-15/environment.json` records execution commit `4402f85f298ae491a70aabd39375b8f0e71e6cd4`, Docker Server 29.1.3, Python 3.10.12, and image identifier `sha256:093ae71c26a28a2c8ef2eca222a7cef4d624ae88638a9a36932223cd270ba982`. The retained `Dockerfile.iperf` uses `ubuntu:22.04` as its base tag; this documents the build recipe rather than an independently captured host or container operating-system version. Exact Windows, WSL, Linux kernel/distribution, and iperf3 versions were not retained in the cohort and are not inferred.

The primary metrics are TCP throughput in Mbps, ping round-trip time (RTT) in milliseconds, end-to-end ping packet loss in percent, and controller reward. Repository-reported (RR) values preserve both the value and interval convention in the retained repository summary. Audit-computed (AC) statistics use complete retained per-run rows, the sample standard deviation, and a two-sided Student’s t 95% confidence interval,

$$\bar{x} \pm t_{0.975, n-1} \frac{s}{\sqrt{n}}.$$

No missing measurements are imputed. For the repeated delay and loss groups, a 200,000-permutation Kruskal–Wallis overall test uses midranks, tie correction, seed 20260821, and the finite-sample estimator $(b+1)/(B+1)$. Epsilon-squared, $(H-k+1)/(N-k)$, reports effect size. This rank-based test avoids relying on normality with $n=5$ and discrete loss values; no pairwise inference is made. The read-only derivative is retained at `derived/audit_computed_statistics/v1_4_inferential_analysis.json`. The RL summary’s intervals use its repository summarizer’s $1.96s/\sqrt{n}$ convention and remain descriptive RR values.

Appendix Tables 4 and 5 fix each experiment’s question, comparator, variables, cohort rule, analysis, and evidence. Bandwidth, Germany50, RL, AI, and Dashboard findings remain descriptive for the dependence, ordering, or sample-size reasons stated there.

4.2 Controlled impairment experiments

The dissertation-primary bandwidth cohort uses direct two-node, server-side token-bucket filtering (TBF) and five-second reverse-TCP iperf3. Its 15 planned runs were executed in fixed blocks—five at 20 Mbps, then five at 50 Mbps, then five at 100 Mbps—under one execution commit, image, and session. All 15 succeeded, so no retry entered the cohort. Per-run configurations, raw iperf3 output, parsed metrics, commands, summary, environment, and cleanup records are retained under `runs/bandwidth-balanced-evidence-supplement-20260826/planned-15/`. The fixed order confounds rate with session time, so intervals and between-rate comparisons remain descriptive. The earlier single 20-Mbps record and earlier 50/100-Mbps supplement remain preserved as independent historical cohorts and are not pooled with this balanced cohort.

Latency used a 20-Mbps routed topology at configured one-way delays of 0, 10, 30, and 50 ms. Each condition has five valid runs and three ping packets per run. The dependent variable is measured RTT, not one-way latency. Packet-loss experiments configured one-way loss of 0, 1, 3, and 5%, with five runs and 100 ping packets per run. Sent and received counts determine measured end-to-end loss. The dual-router benchmark used client–router1–router2–server, zero configured delay/loss, and five runs; it is analyzed separately from direct bandwidth evidence.

4.3 Topology, AI, control, and interface experiments

Three Germany50 path cases were fixed before execution: one, four, and nine backbone hops. Every link had 20 Mbps, 10-ms one-way delay, and 0% loss, with three runs per case. Hop count therefore co-varies with cumulative configured delay; the cases cannot isolate a topology-only causal effect. Appendix Table 7 gives the paths. A separate full-topology evaluation instantiated 50 nodes and 88 links. Selected routing installed 20 routes and produced nine measurements; full routing dry-run validated 4,224 entries without installing or exercising them as all-pairs traffic.

AI validation distinguishes the official OpenAI attempt from a separate OpenAI-compatible Qwen path. The official request produced HTTP 429 `insufficient_quota`. The Qwen-compatible path produced a six-node scenario that passed schema, semantic, prompt-constraint, projection, dry-run, and controlled real-Docker validation.

The RL experiment is a bounded integration and evaluation of closed-loop path selection, not a new RL algorithm or a general RL evaluation. All four policies use the same Docker execution, route-verification, measurement, and evidence pipeline, but after initialization only Q-learning uses prior measured state and updates a Q-table; the heuristic uses the known phase configuration and fixed A/B use fixed paths. The comparison retains 20 valid real-Docker rows per policy. Q-learning rows 1–4 are migrated completed observations; rows 5–20 were produced by a reinitialized `QLearningAgent` without restoring or replaying the earlier Q-table. Its 20 rows are therefore valid descriptive Docker measurements, not one uninterrupted learning trajectory. The single-seed ordered observations are not independent randomized replications, and RR summaries do not establish policy superiority. Two failed/retried checkpoints remain excluded but visible. Appendix Table 8 reports all policies. The Dashboard is one documented direct user-interface (UI) path smoke; absent raw validation logs determine its documented-only status.

5 Results

5.1 Controlled impairments

Figure 2 and Table 1 report the main impairment results. The figure is a dissertation-side composite regenerated from retained evidence; the project-generated delay and loss audit SVGs were checked but omit the balanced bandwidth cohort and interval information. At 20 Mbps, the five valid measurements were all 19.2 Mbps (standard deviation (SD) 0, 95% confidence interval (CI) ± 0 , range 19.2–19.2), a -4.00% deviation. The 50-Mbps block averaged 47.800 Mbps (SD 0.100, 95% CI ± 0.124 , range 47.7–47.9), a -4.40% deviation. The 100-Mbps block averaged 94.760 Mbps (SD 0.913, 95% CI ± 1.133 , range 93.2–95.5), a -5.24% deviation. These are repository-reported descriptive statistics from the 15/15 valid fixed-block cohort. Earlier bandwidth cohorts are retained separately and are not pooled; the dual-router benchmark is also excluded from this direct-topology panel.

Mean RTT increased from 0.105 ms at 0-ms configured one-way delay to 27.008, 80.303, and 133.686 ms at 10, 30, and 50 ms. The corresponding AC sample standard deviations (SDs) were 0.019, 0.056, 0.057, and 0.037 ms. The overall permutation Kruskal–Wallis test gave $H = 17.857$, permutation $p < 10^{-4}$, and $\epsilon^2 = 0.929$. This is an overall association, not pairwise evidence. These observations are RTTs, not measured one-way latency.

Configured one-way loss of 0, 1, 3, and 5% produced mean end-to-end ping loss of 0, 3, 6, and 10%. The model-based round-trip references $100[1 - (1 - p/100)^2]$ are 0, 1.99, 5.91, and 9.75%; they are not an experimental baseline. Across five runs per condition, the observed SDs were 0, 2.345, 3.240, and 3.391 percentage points. The overall permutation test gave $H = 15.611$, permutation $p < 10^{-4}$, and $\epsilon^2 = 0.788$. No corrected pairwise test was performed. Aggregate received/sent counts were 500/500, 485/500, 470/500, and 450/500.

The separate dual-router benchmark completed five valid runs with a mean throughput of 19.220 Mbps, SD 0.045, 95% CI ± 0.056 , and range 19.2–19.3 Mbps. The mean is RR; dispersion and range are AC.

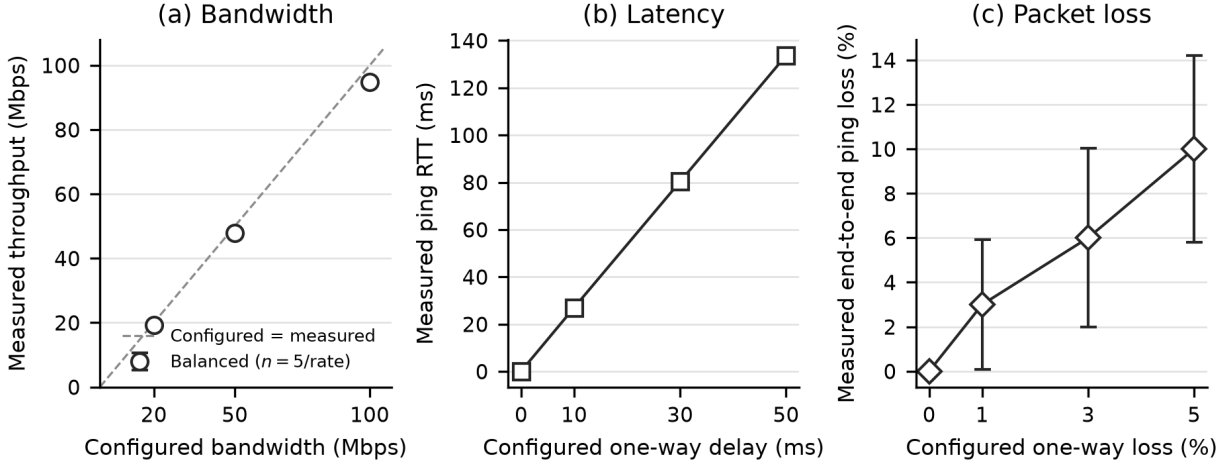


Figure 2: Main impairment results. Bandwidth uses the balanced fixed-block cohort with five runs at each configured rate; the blocks ran in 20/50/100-Mbps order. Latency uses five runs per configured one-way delay; packet loss uses five runs and 100 packets per run. Markers show cohort means: bandwidth uses repository-reported 95% intervals, whereas latency and loss use two-sided Student’s t audit-computed 95% intervals.

Table 1: Main impairment results. BW denotes bandwidth; CI denotes the 95% confidence-interval half-width; RR and AC denote repository-reported and audit-computed values.

Condition	n	Mean	SD	95% CI	Range	Deviation/source
BW 20 Mbps	5	19.200	0.000	± 0.000	19.200–19.200	–4.00%; RR
BW 50 Mbps	5	47.800	0.100	± 0.124	47.700–47.900	–4.40%; RR
BW 100 Mbps	5	94.760	0.913	± 1.133	93.200–95.500	–5.24%; RR
Delay 0 ms	5	0.105	0.019	± 0.024	0.080–0.123	RTT; RR/AC
Delay 10 ms	5	27.008	0.056	± 0.070	26.943–27.089	RTT; RR/AC
Delay 30 ms	5	80.303	0.057	± 0.071	80.249–80.393	RTT; RR/AC
Delay 50 ms	5	133.686	0.037	± 0.046	133.637–133.723	RTT; RR/AC
Loss 0%	5	0.000	0.000	± 0.000	0–0	end-to-end; RR/AC
Loss 1%	5	3.000	2.345	± 2.912	1–7	end-to-end; RR/AC
Loss 3%	5	6.000	3.240	± 4.023	2–9	end-to-end; RR/AC
Loss 5%	5	10.000	3.391	± 4.211	5–14	end-to-end; RR/AC

Bandwidth uses one fixed-block session ($n = 5$ per rate); its intervals are descriptive and older cohorts are not pooled. Units are Mbps for bandwidth, ms for RTT, and percent for loss.

5.2 Topology scaling and Germany50

Figure 3 separates simple execution, the dual-router benchmark, and full Germany50 scope. The one-, four-, and nine-hop cases produced mean RTTs of 41.196, 104.623, and 214.425 ms and throughputs of 18.8, 17.8, and 15.7 Mbps. Appendix Table 7 gives path detail and dispersion. Hop count and cumulative configured per-link delay co-vary, so these descriptive cases do not isolate a topology-only effect.

The full topology preflight recorded 50 nodes, 88 networks, and a theoretical 4,224-route plan. Selected mode installed 20 routes and completed three ping/iperf3 measurements for each representative pair (nine total). The full 4,224-entry plan separately returned dry-run success. No retained evidence shows that all entries were installed or that all node pairs exchanged real traffic.

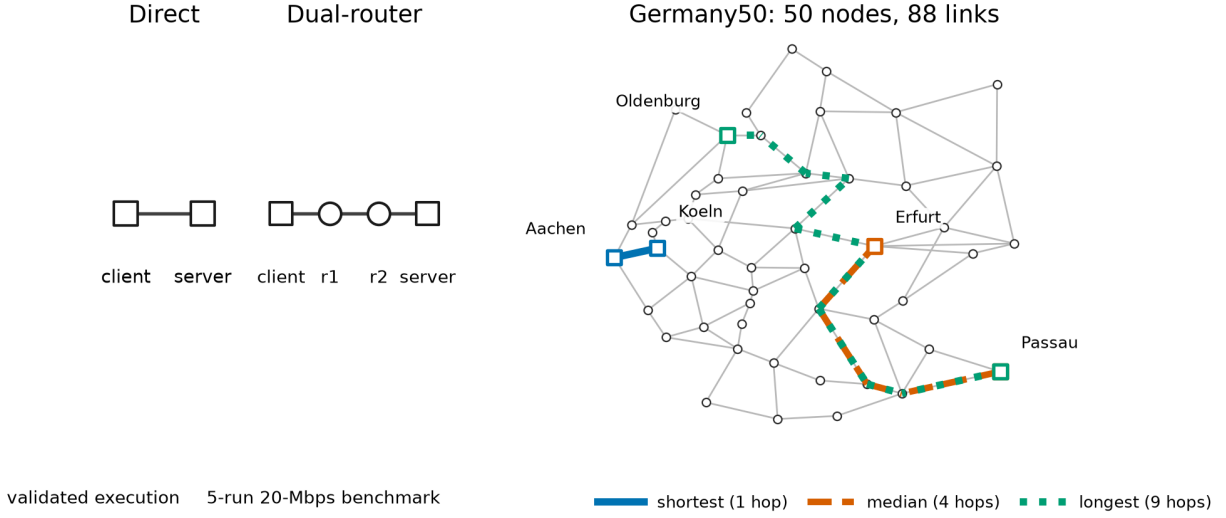


Figure 3: Topology scaling from direct and dual-router configurations to the real 50-node/88-link Germany50 graph. Highlighted paths are the shortest, median, and longest representative paths used for real traffic. The 4,224-entry full route plan was validated in dry-run mode only; the figure does not represent all-pairs traffic.

Table 2: Extended validation and evidence boundaries.			
Component	Evidence	Result	Boundary
Dual-router	Five Docker runs	19.220 Mbps mean; SD 0.045; CI ± 0.056	Separate multi-router benchmark; not part of a direct cohort.
Germany50	Full instantiation; route summaries; nine measurements	50 nodes/88 links; 4,224-entry plan passed dry-run; three paths carried traffic	Dry-run is not route installation; traffic was representative, not all-pairs.
AI	Sanitized official result; compatible-provider run	Official OpenAI: HTTP 429. Qwen-compatible: success	Providers remain separate; Qwen validated success is not official OpenAI success.
RL	Per-episode CSV; policy summary	80 valid Docker measurement rows; mean reward heuristic vs 4.143 Q-learning	Higher heuristic mean applies only to the evaluated setup.
Dashboard	Validation document; implementation	Documented smoke: 0.118-ms mean RTT; 19.4 Mbps	Raw UI/test/security/secret/fresh-clone logs were not retained.

5.3 AI, RL-based closed-loop control, and Dashboard

Table 2 summarizes the extensions. The official OpenAI attempt returned Hypertext Transfer Protocol (HTTP) 429 with `insufficient_quota`; it generated no successful official scenario. The separate Qwen-compatible request produced a validated scenario and a single controlled real-Docker result of 82.194-ms RTT, 18.3-Mbps throughput, and 0% measured loss. No SD or CI is available for this single validation.

All four policies reused the same Docker execution, route-verification, measurement, and evidence pipeline. After initialization, only Q-learning used prior measured RTT, loss, throughput, and current-path state to update a Q-table; the heuristic selected from the known phase configuration, while fixed A/B remained fixed. The threshold heuristic achieved RR mean reward 17.116 (RR SD 0.325 and RR 95% interval half-width ± 0.142), mean RTT 30.189 ms, 0% mean loss, and 18.625-Mbps mean throughput. Q-learning achieved RR mean reward 4.143 (RR SD 15.045 and RR interval half-width ± 6.594), mean RTT 89.225 ms, 2.5% loss, and 9.855-Mbps throughput. These summaries are descriptive. Q-learning selected path A in all 20 recorded

rows and showed no path switch after the phase boundary; the heuristic selected A for ten rows, B for ten, and switched once. The Q-learning rows span a migration and agent reinitialization, so they are not an uninterrupted learning trajectory. These single-seed ordered observations are not independent randomized replications and do not support RL superiority. Appendix Table 8 gives the full summary.

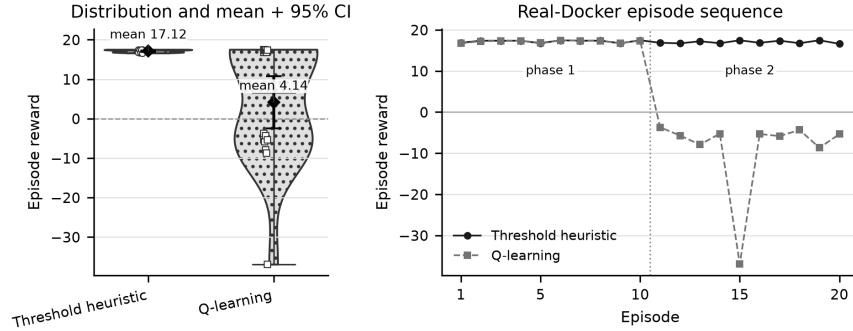


Figure 4: Reward comparison based on 20 retained real-Docker records per displayed policy. The supplement contains 80 valid measurements across four policies. The Q-learning records do not form an uninterrupted trajectory. The threshold heuristic had a higher mean reward in this setup.

The Dashboard validation document records direct run `3cada000bb4d`: three of three ping replies, RTT min/mean/max 0.101/0.118/0.143 ms, 19.4-Mbps throughput, HTTP 200 for JSON/CSV/log/ZIP downloads, and cleanup pass. The raw UI-run, 69-test, security, secret-scan, and fresh-clone logs are not retained, so this result remains documented-only.

6 Discussion

6.1 RQ1: controllability and repeatability

Answer. The workflow enforced configured conditions that were associated with measured bandwidth, RTT, and loss changes, without making configured and measured quantities equivalent.

Evidence. Delay and loss each showed an overall association in permutation tests, with $\epsilon^2 = 0.929$ and 0.788 respectively. Loss means were 0, 3, 6, and 10% against model references of 0, 1.99, 5.91, and 9.75%, but $n = 5$ gives wide uncertainty and no pairwise inference. In the balanced cohort, mean throughput was 19.20, 47.80, and 94.76 Mbps at configured 20, 50, and 100 Mbps, with five valid runs per rate.

Interpretation. TBF sets a token rate, whereas iperf3 observes end-to-end TCP throughput. Likewise, qdisc delay and loss are one-way interface settings, whereas ping reports a bidirectional path. Protocol behaviour, queue placement, scheduling, and shared-host load explain why enforcement and observation must remain separate constructs.

Relation to literature. This result follows the `netem` warning that timers, queues, and protocol behaviour affect observations (iproute2 Project, n.d.). It also gives an empirical instance of the fidelity-efficiency trade-off identified for NDT models (Hui et al., 2023): a lightweight twin exposes controlled trends without establishing physical-network fidelity.

Limitation. Most cohorts contain five runs, and packet-loss intervals are wide. Bandwidth ran in fixed 20/50/100-Mbps blocks within one session, so rate co-varies with execution order and time. The evidence supports the tested ranges on one Docker/WSL host only.

6.2 RQ2: topology extension and routing

Answer. The validated pipeline extends from a small routed case to two routers and to Germany50 structural scale, while large-topology real traffic remains representative rather than exhaustive.

Evidence. Five dual-router runs carried near-20-Mbps traffic. The extracted one-, four-, and nine-hop paths each carried three measured flows. The full 50-node/88-link graph was instantiated; selected mode installed 20 routes for three endpoint pairs and produced nine measurements, while the 4,224-entry full plan passed dry-run only.

Interpretation. The small topology tests execution, the dual-router case tests explicit forwarding, and Germany50 tests graph construction, route planning, and selected-path operation. In the selected paths, hop count and cumulative configured delay co-vary, so RTT and throughput differences cannot be attributed to hop count alone. Dry-run validates structure and command planning, not runtime installation or all-pairs correctness.

Relation to literature. Topology Zoo contributes realistic structural inputs rather than execution evidence (Knight et al., 2011). The results connect that structural resource to a container experiment while retaining the distinction, central to reproducible emulation (Handigol et al., 2012), between a runnable specification and the traffic actually observed.

Limitation. Only three Germany50 endpoint pairs carried real traffic, and path factors are confounded. The study therefore makes no all-pairs, wide-area, or operational-scale claim.

6.3 RQ3: AI and RL-based closed-loop extensibility

Answer. Guarded generation and measured closed-loop control can reuse the workflow, but provider availability and controller performance limit the result. RL serves only as a bounded closed-loop path-selection use case: it tests whether measured RTT, loss, and throughput can feed a controller through the shared execution-and-evidence pipeline. The contribution lies in integrating and evaluating RL-based closed-loop control within the shared pipeline, rather than proposing a new RL algorithm or claiming superiority over the heuristic.

Evidence. The Qwen-compatible scenario passed validation and one controlled execution; the official OpenAI request returned HTTP 429. The retained comparison contains 80 valid Docker measurement rows across four policies. The threshold heuristic switched at the phase boundary and had a higher descriptive mean reward. Q-learning selected path A in all 20 recorded Q-learning rows and showed no path switch after the boundary.

Interpretation. Projection and validation constrain generated text before execution, so the observation demonstrates a guarded integration path rather than trust in an LLM. The heuristic encoded the phase boundary directly. The recorded Q-learning behaviour is consistent with an insufficient response under this formulation, but does not identify its cause.

Relation to literature. The validation boundary responds to reliability and security concerns in LLM-assisted networking (Lira et al., 2025; Liu et al., 2025). Digital-twin-assisted RL can improve learning in other designs (Güemes-Palau et al., 2022; Cheng et al., 2025), but those results do not guarantee superiority in this low-dimensional ordered task.

Limitation. One compatible-provider record is not official OpenAI success. Q-learning rows 1–4 were migrated observations, while rows 5–20 used a reinitialized agent without restoration or replay of the earlier Q-table. The 20 valid measurement rows therefore do not form an uninterrupted learning trajectory. One seed, fixed hyperparameters, and this ordered formulation cannot show whether another duration or

formulation would switch paths; rows are not independent randomized replications. The Dashboard is only a supporting interface.

6.4 Threats to validity and limitations

Internal validity. Experiments share a host, Docker engine, kernel, and virtual networking stack. Uncontrolled host load and scheduling can influence throughput and timing. The retained runners, route/qdisc checks, and repeated cohorts reduce but do not remove this risk. The balanced bandwidth cohort completed all 15 planned runs without retry, but its fixed block order does not separate rate from session time.

Construct validity. Configured one-way delay and loss are not the same constructs as measured RTT and end-to-end ping loss. TCP throughput is affected by more than the configured TBF rate. Reward is specific to the retained RL formulation and should not be generalized to all network objectives.

Statistical conclusion validity. Delay and loss have overall rank-based permutation tests, but $n = 5$ per group limits precision and no pairwise inference is claimed. Bandwidth intervals are descriptive and its blocks are ordered; Germany50 is confounded; RL measurement rows are ordered and dependent; Qwen and Dashboard are single records. These designs do not support additional significance claims.

External validity. Results apply to the evaluated Docker/WSL environment, topologies, traffic tools, and impairment ranges. Equivalence with hardware routers, production workloads, other kernels, or wide-area networks was not tested. Germany50 represents structural scale but only three real traffic paths. The Q-learning result applies to one state/action/reward design and one hyperparameter setting.

Reproducibility and evidence preservation. Scenarios, raw metrics, summaries, scripts, plots, failures, and cleanup records provide substantial provenance. Some statistics in this draft are AC derivatives and remain separate from repository evidence. Later reviewer-readiness checks establish a practical fresh-clone path and 69 passing unit tests, but they post-date the formal experiments and do not increase their sample sizes or coverage. Raw historical Dashboard smoke logs remain unavailable.

Overall, the main RQ has a qualified positive answer within the tested scope: the pipeline enforced and measured controlled conditions and was reused for multi-router, Germany50, guarded-generation, and measured-control cases. It provides a reproducible testbed for controlled container-network experiments before hardware or operational testing, but does not establish production behaviour, physical-network fidelity, exhaustive topology traffic, provider reliability, or learning-policy superiority.

7 Conclusion

The main question asked to what extent a bounded Docker NDT can enforce and measure controlled conditions while reusing one validated pipeline across topology and guarded automation extensions. The answer is conditional but positive within the tested configurations. For RQ1, delay and loss had significant overall associations with measured RTT and loss, while bandwidth measurements showed that configured rates are reference targets rather than end-to-end values. For RQ2, the pipeline extended through two routers to a 50-node/88-link Germany50 instantiation; the 4,224-entry plan was dry-run validated, and real traffic covered three representative, confounded path cases. For RQ3, guarded Qwen-compatible generation and 80 measured control rows reused the pipeline; official OpenAI returned HTTP 429, and Q-learning showed no path switch across its 20 single-seed rows. Those Q-learning rows span a migration and agent reinitialization rather than one uninterrupted learning trajectory.

The contribution is a reproducibility-oriented experimental integration, not a validated replica of a physical network. Shared-host effects, small cohorts, representative Germany50 coverage, provider availability, historical Dashboard evidence gaps, and one learner formulation bound the conclusion. Traceability remains a methodology and audit property, not an independently quantified success claim. Future work should test independent hosts and broader traffic coverage while preserving the sealed evidence.

References

- Jeff Ahrenholz, Claudiu Danilov, Thomas R. Henderson, and Jae H. Kim. CORE: A real-time network emulator. In *MILCOM 2008—2008 IEEE Military Communications Conference*, pp. 1–7. IEEE, 2008. doi: 10.1109/MILCOM.2008.4753614. URL <https://doi.org/10.1109/MILCOM.2008.4753614>.
- Paul Almasan, Miquel Ferriol-Galmés, Jordi Paillisse, José Suárez-Varela, Diego Perino, Diego López, Antonio Agustin Pastor Perales, Paul Harvey, Laurent Ciavaglia, Leon Wong, Vishnu Ram, Shihan Xiao, Xiang Shi, Xiang Cheng, Albert Cabellos-Aparicio, and Pere Barlet-Ros. Network digital twin: Context, enabling technologies, and opportunities. *IEEE Communications Magazine*, 60(11):22–27, November 2022. doi: 10.1109/MCOM.001.2200012. URL <https://doi.org/10.1109/MCOM.001.2200012>.
- Nan Cheng, Xiucheng Wang, Zan Li, Zhisheng Yin, Tom H. Luan, and Xuemin Shen. Toward enhanced reinforcement learning-based resource management via digital twin: Opportunities, applications, and challenges. *IEEE Network*, 39(1):189–196, 2025. doi: 10.1109/MNET.2024.3438543. URL <https://doi.org/10.1109/MNET.2024.3438543>.
- Sebastian Gallenmüller, Dominik Scholz, Henning Stubbe, and Georg Carle. The pos framework: A methodology and toolchain for reproducible network experiments. In *Proceedings of the 17th International Conference on Emerging Networking Experiments and Technologies*, pp. 259–266. Association for Computing Machinery, 2021. doi: 10.1145/3485983.3494841. URL <https://doi.org/10.1145/3485983.3494841>.
- Carlos Güemes-Palau, Paul Almasan, Shihan Xiao, Xiang Cheng, Xiang Shi, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Accelerating deep reinforcement learning for digital twin network optimization with evolutionary strategies. In *NOMS 2022—2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–5. IEEE, 2022. doi: 10.1109/NOMS54207.2022.9789945. URL <https://doi.org/10.1109/NOMS54207.2022.9789945>.
- Nikhil Handigol, Brandon Heller, Vimalkumar Jeyakumar, Bob Lantz, and Nick McKeown. Reproducible network experiments using container-based emulation. In *Proceedings of the 8th International Conference on Emerging Networking Experiments and Technologies*, pp. 253–264. Association for Computing Machinery, 2012. doi: 10.1145/2413176.2413206. URL <https://doi.org/10.1145/2413176.2413206>.
- Linbo Hui, Mowei Wang, Liang Zhang, Lu Lu, and Yong Cui. Digital twin for networking: A data-driven performance modeling perspective. *IEEE Network*, 37(3):202–209, May 2023. doi: 10.1109/MNET.119.2200080. URL <https://doi.org/10.1109/MNET.119.2200080>.
- International Telecommunication Union. Digital twin network—requirements and architecture. Recommendation ITU-T Y.3090, ITU Telecommunication Standardization Sector, February 2022. URL <https://www.itu.int/rec/T-REC-Y.3090/en>.
- iproute2 Project. *tc-netem(8): Network Emulator*, n.d. URL <https://man7.org/linux/man-pages/man8/tc-netem.8.html>. Linux manual page; accessed 2026-08-10.
- Simon Knight, Hung X. Nguyen, Nickolas Falkner, Rhys Bowden, and Matthew Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 29(9):1765–1775, October 2011. doi: 10.1109/JSAC.2011.111002. URL <https://doi.org/10.1109/JSAC.2011.111002>.
- Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: Rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, pp. 1–6. Association for Computing Machinery, 2010. doi: 10.1145/1868447.1868466. URL <https://doi.org/10.1145/1868447.1868466>.
- Oscar G. Lira, Oscar M. Caicedo, and Nelson L. S. da Fonseca. Large language models for zero touch network configuration management. *IEEE Communications Magazine*, 63(7):146–153, 2025. doi: 10.1109/MCOM.001.2400368. URL <https://doi.org/10.1109/MCOM.001.2400368>.
- Chang Liu, Xiaohui Xie, Xinggong Zhang, and Yong Cui. Large language models for networking: Workflow, advances, and challenges. *IEEE Network*, 39(5):165–172, September 2025. doi: 10.1109/MNET.2024.3510936. URL <https://doi.org/10.1109/MNET.2024.3510936>.

-
- Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, pp. 50–56. Association for Computing Machinery, 2016. doi: 10.1145/3005745.3005750. URL <https://doi.org/10.1145/3005745.3005750>.
- Manuel Peuster, Holger Karl, and Steven van Rossem. MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments. In *2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*, pp. 148–153. IEEE, 2016. doi: 10.1109/NFV-SDN.2016.7919490. URL <https://doi.org/10.1109/NFV-SDN.2016.7919490>.
- Mariano Scazzariello, Lorenzo Ariemma, and Tommaso Caiazzi. Kathará: A lightweight network emulation system. In *NOMS 2020—2020 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–2. IEEE, 2020. doi: 10.1109/NOMS47738.2020.9110351. URL <https://doi.org/10.1109/NOMS47738.2020.9110351>.
- Cheng Zhou, Hongwei Yang, Xiaodong Duan, Diego Lopez, Antonio Pastor, Qin Wu, Mohamed Boucadair, and Christian Jacquenet. Network digital twin (NDT): Concepts and reference architecture. Internet-Draft draft-irtf-nmrg-network-digital-twin-arch-13, Internet Research Task Force, July 2026. URL <https://datatracker.ietf.org/doc/draft-irtf-nmrg-network-digital-twin-arch/13/>. Work in Progress.

A Appendix

A.1 Detailed experiment and provenance notes

Table 3 expands the main experimental matrix with evidence locations. Paths are relative to the read-only evidence repository unless prefixed by `derived/`, which denotes a dissertation-side audit derivative.

The dissertation-side consolidated inventory is `notes/experiment_inventory.md`. Its machine-readable counterpart is `derived/audit_computed_statistics/experiment_summary.csv`. RR values reproduce repository summaries; AC values were computed from retained per-run data without changing the evidence repository.

Table 3: Detailed evidence index for the principal experiments.		
Experiment	Raw or primary evidence	Summary / reproduction
Primary 20/50/100	BW runs/bandwidth-balanced-evidence-supplement-20260826/planned-15/	summary.json, per-run-results.csv, scripts/ run_bandwidth_evidence_supplement.py
Earlier BW cohorts	runs/current/metrics.json, runs/bandwidth-evidence-supplement/	Retained independently; not pooled with the primary cohort
Latency	runs/delay_batch_20260713-135915/delay_*_run_*/metrics.json	runs/final-evaluation/ evidence-inventory.json, scripts/run_delay_batch.py
Packet loss	runs/packet_loss_batch_20260713-153303/ loss_*_run_*/metrics.json	runs/final-evaluation/ evidence-inventory.json, scripts/run_packet_loss_batch.py
Dual-router	runs/two_router_batch_20260713-160810/ baseline_run_*/metrics.json	scripts/run_two_router_batch.py
Germany50 paths	runs/germany50-selected-paths-final/ *-run-*/metrics.json	runs/germany50-selected-paths-final/ summary.json, runs/germany50-selected-paths-final/ summary.csv, scripts/ run_germany50_selected_path_tests.py
Germany50 full	runs/final-evaluation/ germany50-full-fixed/ selected-attempt-1/	runs/final-evaluation/ germany50-full-fixed/ route-plan-dry-run/ summary.json, scripts/ run_germany50_full_evaluation.py
Official OpenAI	Sanitized source reference only	runs/final-evaluation/ ai-openai-429-evidence.json
Qwen-compatible	runs/openai-compatible-live-validation-20260714-064525/	runs/final-evaluation/ ai-qwen-live-summary.json
RL	runs/final-evaluation/ rl-docker-supplement/ per-episode-results.csv	runs/final-evaluation/ rl-docker-supplement/ summary.json, runs/ final-evaluation/ rl-docker-supplement/ policy-summary.csv, scripts/ run_rl_docker_supplement.py
Dashboard	Raw validation logs not retained	docs/final/ interactive-dashboard-validation.md

Table 4: RQ1 analysis matrix. A configured token rate is a reference target, not an experimental baseline.

Case	Control/reference variable and Outcome	Valid/excluded	Analysis/evidence
BW20	No separate baseline; TCP rate configured token rate is target	5/0	RR mean, SD, CI, range; balanced bandwidth
BW50	No separate baseline; TCP rate configured token rate is target	5/0	RR mean, SD, CI, range; balanced bandwidth
BW100	No separate baseline; TCP rate configured token rate is target	5/0	RR mean, SD, CI, range; balanced bandwidth
Delay	0-ms control; RTT 0/10/30/50-ms one-way delay	5 each/0	Overall permutation test, epsilon-squared, AC intervals; latency
Loss	0% control; 0/1/3/5% End-to-end loss one-way loss; model round-trip reference	5 each/0	Overall permutation test, epsilon-squared, AC intervals/counts; loss

Table 5: RQ2/RQ3 and interface analysis matrix. These cases remain descriptive.

Case	Control/reference variable and Outcome	Valid/excluded	Analysis/evidence
Dual router	Zero impairment; fixed TCP rate topology	5/0	RR/AC descriptive; dual-router
Germany paths	Representative path case; RTT; TCP rate hops and cumulative delay co-vary	3 each/0	Confounded case comparison; Germany50 paths
Germany full AI	Selected execution vs full-plan dry-run Provider/validation path; no performance baseline	Route/traffic counts and one-run metrics	9 traffic + 1 dry-run/0
Control	Fixed A/B and heuristic comparators; policy	Validation and net-80/2 retries work metrics	1 + 1/0
Dashboard	No baseline; one direct HTTP, files, met-scenario	1 documented/0	Documented-only descriptive record

A.2 Closest-work capability comparison

Table 6: Critical comparison with the closest emulation and reproducibility work.

Work	Topology, containers, im- pairment	Reproducibility and scale	Difference from this RQ
Mininet/ Handigol	Namespace topologies, real stacks, link control	Runnable experiments and retained artifacts	Strong precedent; not this Docker cross-extension evalu- ation
CORE	Real-time wired/wireless and distributed emulation; physical links	More than 100 virtual nodes reported	Broad emulator; not Docker- specific evidence/automation integration
Containernet/ MeDICINE	Docker hosts/functions, link control, multi-PoP	Dynamic orchestration and service prototyping	Broader container orchestra- tion; different evaluation ob- jective
Kathará	Container network devices and complex scenarios	Comparative perfor- mance/scalability evidence	Broader emulation capability; not the evaluated evidence pipeline
pos	Orchestration, measure- ment, and evaluation	Automated artifact bun- dles and repeatable work- flow	Closest reproducibility toolchain; not this NDT- specific study
This work	Bounded Docker scenarios; route/qdisc checks	Germany50 structure, se- lected paths, per-run arti- facts	Quantifies one validated pipeline only in the tested scope

A.3 Implementation specification

This compact JSON is valid under the generic project schema:

```
{
  "topology_name": "client-router-server",
  "addressing": {
    "base_cidr": "10.64.0.0/16",
    "subnet_prefixlen": 29
  },
  "nodes": [
    { "id": "client", "type": "client" },
    { "id": "router", "type": "router" },
    { "id": "server", "type": "server" }
  ],
  "links": [
    {
      "source": "client", "target": "router",
      "bandwidth_mbps": 20, "delay_ms": 0, "packet_loss_percent": 0
    },
    {
      "source": "router", "target": "server",
      "bandwidth_mbps": 20, "delay_ms": 10, "packet_loss_percent": 0
    }
  ],
  "traffic": {
    "source": "client", "destination": "server", "protocol": "tcp",
    "duration_s": 5, "ping_count": 3, "reverse": true
  },
  "route_generation_mode": "traffic_endpoint_subnets"
}
```

Validation. The generic path requires documented top-level fields; known link and traffic endpoints; rejection of duplicate undirected links and disconnected topologies; duplicate interface-address and route-conflict checks; non-negative delay; loss within 0–100%; and positive bandwidth. The guarded AI schema is narrower: 2–10 nodes, lower-case identifiers beginning with a letter (maximum 32 characters), exactly one client and one server, TCP only, delay 0–250 ms, loss 0–20%, bandwidth 1–1000 Mbps, duration 1–30 s, and ping count 1–10. It also rejects command/Internet Protocol (IP)-shaped strings, checks prompt constraints and connectivity, and projects only allowlisted fields.

Addressing and routing. Generic execution uses 10.64.0.0/16 and assigns successive /29 point-to-point subnets. The bridge reserves the first usable address; link endpoints receive the next two. Breadth-first search with sorted neighbours generates deterministic shortest paths. Static routes target either all unattached

subnets or traffic-endpoint subnets; equal-length candidates use lexicographic path order. Runtime verification executes `ip route show <destination>` and checks `<destination> via <next-hop>`. Germany50 selected mode covers retained endpoint paths; full mode is a generated dry-run plan.

Traffic control and measurement. Bandwidth-only links execute:

```
tc qdisc replace dev <iface> root tbf rate <X>mbit \
    burst 32kbit latency 400ms
```

For bandwidth combined with delay and/or loss, `build_combined_qdisc_commands` instead emits one root `netem` command containing applicable `delay <D>ms`, `loss <L>%`, and `rate <X>mbit` terms. Impairment-only links use root `netem`. Measurements use `ping -c <N> <destination>` and `iperf3 -c <destination> -R -t <seconds>`. The runner parses output, writes run-scoped artifacts, verifies routes/qdiscs where supported, and removes only project-owned run resources.

A.4 Configuration and measurement details

The dissertation-primary direct bandwidth cohort used five-second reverse TCP iperf3 with server-side TBF. Its fixed blocks ran at 20, 50, and 100 Mbps in that order, with five planned and five valid runs per block under execution commit `4402f85f298ae491a70aab39375b8f0e71e6cd4` and one retained image/session record. No retry was required. Raw iperf3 output, parsed metrics, command logs, environment, summary, and cleanup are retained. The earlier single 20-Mbps record and 50/100-Mbps supplement remain separate historical cohorts and are not pooled.

Latency used configured one-way delays of 0/10/30/50 ms with five runs and three ping packets per run. Packet loss used configured one-way probabilities of 0/1/3/5% with five runs and 100 packets per run. For both experiments, measured end-to-end quantities are reported separately from configured one-way impairments.

A.5 Germany50 selected paths

Table 7: Descriptive Germany50 path-case comparison. Hop count and cumulative configured per-link delay co-vary, so the table does not estimate an isolated hop-count effect.

Class	Backbone nodes	Hops	RTT mean	RTT SD	Rate mean
Shortest	Aachen–Koeln	1	41.196	0.259	18.8
Median	Erfurt–Wuerzburg–Augsburg–Muenchen– Passau	4	104.623	0.845	17.8
Longest	Oldenburg–Bremen–Hannover– Braunschweig–Kassel–Erfurt–Wuerzburg– Augsburg–Muenchen–Passau	9	214.425	0.441	15.7

Each extracted link used 20 Mbps, 10-ms configured one-way delay, and 0% configured loss. These are path-extracted experiments, not full-topology deployments. The separate full-topology selected-mode run installed 20 routes for the three endpoint pairs. The 4,224-entry plan was dry-run validated only.

A.6 RL policy settings and complete policy summary

Actions are paths A and B. Each agent invocation starts from initialized synthetic path-A metrics; thereafter only Q-learning uses state comprising the prior measured RTT (low below 45 ms, otherwise high), loss (none below 1%, otherwise loss), throughput (high at or above 16 Mbps, otherwise low), and current path, then updates its Q-table from measured reward. The heuristic selects the path minimizing configured delay plus four times configured loss in the known phase; fixed A/B always select their named paths. Reward is:

```
R = throughput_mbps - 0.05 * rtt_ms
    - 0.50 * packet_loss_percent
```

In Phase 1, A has (5 ms, 0%, 20 Mbps) and B has (30 ms, 4%, 12 Mbps); Phase 2 swaps these conditions. The schedule is ordered, not randomized. Q-learning uses $\alpha = 0.35$, $\gamma = 0.85$, $\epsilon = 0.18$, and seed 20260722. Its completed rows 1–4 were migrated; rows 5–20 used a newly initialized `QLearningAgent`, with no restoration or replay of the earlier Q-table. Each policy has 20 valid descriptive Docker measurement rows, but the Q-learning rows are not one uninterrupted learning trajectory. Failed/retried checkpoints remain in evidence and are excluded from valid-row statistics.

Table 8: Descriptive repository-reported real-Docker policy summary from one seed and an ordered two-phase schedule. Rows are not independent randomized policy replications; Q-learning rows span a migration and agent reinitialization.

Policy	n	Reward mean	Reward SD	RTT (ms)	Loss (%)	Rate (Mbps)
Q-learning	20	4.143	15.045	89.225	2.500	9.855
Threshold heuristic	20	17.116	0.325	30.189	0.000	18.625
Fixed A	20	-1.419	21.126	99.609	12.500	9.812
Fixed B	20	0.257	19.665	91.027	10.000	9.808

The RR reward confidence-interval (CI) half-widths were 6.594, 0.142, 9.259, and 8.619 in row order. They preserve the repository summarizer’s $1.96s/\sqrt{n}$ convention as a descriptive summary. A single seed and ordered measurement rows do not supply independent randomized replications or an inferential policy comparison.

A.7 AI and Dashboard boundaries

The AI execution chain comprises structured generation, schema validation, semantic validation, prompt-constraint checking, projection to an executable scenario, dry-run, and controlled Docker execution. Official OpenAI returned HTTP 429 `insufficient_quota`; a separate Qwen-compatible provider completed validation. The provider distinction must be preserved in any extracted result or presentation.

The Dashboard supports bounded small-topology interaction. Its documented run `3cada000bb4d` used 20 Mbps, zero delay/loss, three pings, and three seconds of `iperf3`. Documentation reports 3/3 replies, 0.101/0.118/0.143-ms RTT, 19.4 Mbps, successful downloads, and cleanup. Raw UI-path, 69-test, security, secret-scan, and fresh-clone validation logs were not retained, so these checks are not presented as independently auditable evidence.

A.8 Reproduction boundary

The final code/evidence publication is GitHub main commit `09a14859bb17`; the link resolves the full immutable SHA. The snapshot includes the dissertation-primary balanced bandwidth cohort listed in Table 3. In a fresh WSL clone with Docker and Python available, the shortest supported path is:

```
bash scripts/reviewer_setup.sh
bash scripts/reviewer_dashboard.sh
# open http://127.0.0.1:8765
```

The setup script creates `.venv-wsl311`, installs the declared requirements, builds `my-iperf-tc`, and runs the 69 unit tests. The start script launches `dashboard/interactive_server.py` on port 8765. Reviewers may inspect retained evidence and use all three small topologies in dry-run mode; real Dashboard execution is intentionally direct-only. Reviewer-readiness checks post-date the original evidence sealing and are not additional formal observations. The separately retained balanced bandwidth cohort was added as a later supplementary replication and does not alter or rerun the sealed delay, loss, Germany50, AI, or RL matrices. A future independent replication should retain host/kernel/Docker versions, image digests, stdout/stderr, and resource utilization, then compare new results without overwriting the evidence used here.

A.9 Generative AI acknowledgement

ChatGPT was used for coding assistance, debugging, language editing, and consistency checking. The author verified the experimental results, retained evidence, citations, and final interpretation and remains responsible for the submitted work.